

Тестовое задание — QA инженер-тестировщик

Позиция: QA инженер-тестировщик

Расчётное время выполнения: 6–8 часов чистого времени

Срок сдачи: 5 рабочих дней с момента получения

Формат сдачи: Ссылка на публичный репозиторий GitHub

Цель задания

Оценить практические навыки кандидата в области тестирования веб-приложений: умение проектировать тест-кейсы, работать с REST API, писать автотесты, формулировать SQL-запросы для проверки данных и документировать процессы в виде диаграмм.

Нас интересует не количество написанного, а осознанность подхода: почему выбраны именно эти сценарии, какие техники применены, как структурирован результат.

Целевые приложения

Для выполнения задания использовать следующие публично доступные стенды:

Стенд	URL	Назначение
Restful Booker	https://restful-booker.herokuapp.com	REST API (задания 1, 4)
The Internet	https://the-internet.herokuapp.com	UI (задания 2, 3)
Документация API	https://restful-booker.herokuapp.com/apidoc	Swagger-описание эндпоинтов

Функциональные требования

Задание 1 — REST API тестирование (Postman)

Покрыть основные сценарии CRUD для ресурса `/booking` Restful Booker API.

Обязательные эндпоинты:

- `POST /auth` — получение токена авторизации
- `GET /booking` — получение списка бронирований с фильтрацией
- `GET /booking/:id` — получение бронирования по ID
- `POST /booking` — создание нового бронирования
- `PUT /booking/:id` — полное обновление бронирования
- `PATCH /booking/:id` — частичное обновление бронирования
- `DELETE /booking/:id` — удаление бронирования

Требования к покрытию:

- Позитивные сценарии для каждого эндпоинта

- Негативные сценарии: невалидные данные, неавторизованный доступ, несуществующий ID, отсутствующие обязательные поля
- Граничные значения для полей дат (`checkin` , `checkout`): корректная дата, прошедшая дата, `checkout` раньше `checkin` , невалидный формат
- Проверка фильтрации `GET /booking` по `firstname` , `lastname` , `checkin` , `checkout`

Требования к коллекции:

- Использовать environment-переменные: `base_url` , `token` , `booking_id`
- Assertions обязательны для каждого запроса: статус-код, структура JSON-ответа (схема или `pm.expect`), время ответа
- Коллекция должна запускаться через Newman без ручных правок: `newman run collection.json -e environment.json`
- Пре-реквест скрипты для подготовки данных там, где необходимо
- В README указать: найденные или потенциальные баги API (минимум 1)

Задание 2 — UI тест-кейсы (ручное тестирование)

Выбрать **2-3 страницы** из the-internet.herokuapp.com и составить тест-кейсы.

Рекомендуемые страницы (или любые другие на усмотрение кандидата):

- Form Authentication (`/login`)
- Drag and Drop (`/drag_and_drop`)
- Dynamic Loading (`/dynamic_loading`)
- File Upload (`/upload`)

Формат тест-кейса:

Поле	Описание
Test ID	Уникальный идентификатор, например <code>TC-LOGIN-001</code>
Title	Краткое название сценария
Priority	High / Medium / Low
Preconditions	Состояние системы перед выполнением
Steps	Пошаговые действия
Expected result	Ожидаемый результат
Actual result	Фактический результат (при ручном прогоне)
Status	Pass / Fail / Blocked

Требования к покрытию:

- Минимум 10 тест-кейсов суммарно по выбранным страницам
- Покрытие: позитивные, негативные, граничные значения
- Применить хотя бы одну технику тест-дизайна — явно указать, какую и почему:
- Boundary Value Analysis (BVA)
- Equivalence Partitioning (EP)
- State Transition Testing
- Pairwise / All-Pairs

- Раздел "Найденные дефекты": если в процессе найдены баги — оформить в виде баг-репортов (Title, Steps to reproduce, Expected / Actual, Severity, Screenshot)

Задание 3 — Автотесты (E2E)

Написать автотесты для **2-3 UI-сценариев** на the-internet.herokuapp.com.

Фреймворк: Playwright, Cypress или Selenium с WebDriver — на выбор кандидата.

Обязательные требования:

- Минимум 5 тест-кейсов: 3 позитивных + 2 негативных
- Архитектура: Page Object Model — обязательно
- Gherkin-описание (`.feature` файлы) для минимум 2 сценариев
- Параметризация хотя бы одного теста (разные входные данные через data-driven подход)
- Тесты должны запускаться из одной README-команды:

```
npm install
npx playwright test      # или npx cypress run / mvn test
```

Рекомендуемая структура проекта:

```
/tests
  /e2e
    login.spec.ts
    upload.spec.ts
  /features
    login.feature
    upload.feature
  /pages
    LoginPage.ts
    UploadPage.ts
  /fixtures
    users.json
  README.md
```

Задание 4 — SQL и NoSQL

4а. SQL-запросы

Дана схема базы данных:

```
Users      (id, name, email, created_at)
Rooms      (id, type, price_per_night)
Bookings   (id, user_id, room_id, checkin, checkout, status, created_at)
status: 'pending' | 'confirmed' | 'cancelled'
```

Написать SQL-запросы для следующих QA-задач:

1. Найти пользователей, у которых нет ни одного бронирования
2. Найти пересекающиеся бронирования одной комнаты (date overlap)
3. Топ-3 комнаты по количеству подтверждённых бронирований за последние 30 дней

4. Найти бронирования со статусом `confirmed`, созданные более 7 дней назад, у которых дата `checkin` уже прошла
5. Количество бронирований по типу комнаты (GROUP BY), только для статуса `confirmed`

Для каждого запроса добавить однострочный комментарий: что именно проверяется и зачем это нужно QA.

4b. NoSQL — письменный ответ (200-300 слов)

Ответить на следующие вопросы:

1. Как бы ты хранил данные о бронированиях в MongoDB? Предложи структуру документа (embedded vs referenced).
2. Какие индексы создал бы на коллекции бронирований и почему?
3. Как проверил бы корректность данных при тестировании через MongoDB Compass или mongo shell?

Задание 5 — Документация и диаграммы

Описать процесс **создания и отмены бронирования** от лица пользователя.

Требуется:

1. **BPMN-диаграмма** — основной happy path + 2 исключения (например: комната недоступна, оплата не прошла). Инструменты: draw.io, Camunda Modeler, Lucidchart.
2. **Sequence Diagram** — взаимодействие компонентов при создании бронирования: User → Frontend → API → Database → Email Service. Инструменты: PlantUML, Mermaid, draw.io.
3. **State Transition** — указать в комментарии к диаграммам (или отдельным разделом), какие переходы состояний объекта `Booking` необходимо покрыть тестами и почему.

Допустимый формат: PNG/SVG-файл + исходник (`.bpmn`, `.puml`, `.drawio`) или Mermaid-код в Markdown.

Технические требования

Репозиторий

- Публичный репозиторий на GitHub
- Структура проекта логична и понятна без дополнительных объяснений

README.md (обязателен)

README должен содержать:

- Краткое описание: что сделано и из чего состоит проект
- Инструкция запуска автотестов (copy-paste ready)
- Инструкция запуска Postman-коллекции через Newman
- Раздел "Найденные баги / наблюдения" (минимум 1 пункт)
- Раздел "Что бы улучшил при наличии дополнительного времени"

Окружение

- Node.js 18+ (для Playwright / Cypress)
- Postman v10+ / Newman для запуска коллекции
- SQL-запросы совместимы с PostgreSQL или MySQL (указать явно)

Критерии оценки

Блок	Критерии	Баллы
REST API + Postman	Полнота покрытия эндпоинтов, качество assertions, наличие негативных кейсов, запускаемость через Newman	25
UI тест-кейсы	Структура тест-кейсов, применение техник тест-дизайна, приоритезация, оформление баг-репортов	20
Автотесты	Page Object Model, Gherkin, параметризация, читаемость кода, запускаемость	25
SQL / NoSQL	Корректность запросов, понимание схемы, качество ответа по MongoDB	15
Диаграммы	Корректность BPMN, полнота Sequence Diagram, State Transition	10
Репозиторий и README	Структура, читаемость, инструкции, раздел с наблюдениями	5
Итого		100

Шкала решений

Баллы	Решение
80-100	Оффер
65-79	Техническое интервью с разбором задания
50-64	Отказ с письменной обратной связью
< 50	Отказ

Бонусные баллы (до +10)

- CI через GitHub Actions: автозапуск тестов на push (+4)
- Allure Report или встроенный HTML-отчёт Playwright (+3)
- Dockerfile для изолированного запуска тестов (+3)

Инструкции по выполнению

Шаг 1 — Подготовка

1. Создать публичный репозиторий на GitHub с названием `qa-middle-assignment`
2. Создать ветку `main` и работать в ней
3. Ознакомиться с документацией Restful Booker API: <https://restful-booker.herokuapp.com/apidoc>

Шаг 2 — Рекомендуемый порядок и тайм-оценки

Задание	Время
Задание 1 (Postman)	~1.5 часа
Задание 2 (UI кейсы)	~1.5 часа
Задание 3 (Автотесты)	~2.5 часа
Задание 4 (SQL/NoSQL)	~0.5 часа
Задание 5 (Диаграммы)	~0.5 часа
README + финальный ревью	~0.5 часа

Шаг 3 — Рекомендуемая структура репозитория

```
/
├─ README.md
├─ postman/
│   └─ collection.json
│   └─ environment.json
├─ test-cases/
│   └─ ui-test-cases.md
├─ tests/
│   └─ e2e/
│   └─ features/
│   └─ pages/
├─ sql/
│   └─ queries.sql
├─ nosql/
│   └─ mongodb-answer.md
└─ diagrams/
    └─ booking-bpmn.png
    └─ sequence.puml
    └─ sequence.png
```

Шаг 4 — Сдача

1. Убедиться, что репозиторий публичный
2. Проверить: `npm install && npx playwright test` проходит без ошибок на чистой машине
3. Проверить: Newman запускает коллекцию без ручных правок
4. Отправить ссылку на репозиторий ответным письмом

Важные замечания

- **Не нужно покрывать всё подряд** — важна осознанность выбора сценариев, а не их количество
- **Комментируй решения** — если применил технику тест-дизайна, укажи это явно
- **Баги — это хорошо** — если нашёл нестандартное поведение на стендах, задокументируй, это плюс
- **Вопросы приветствуются** — если что-то непонятно в задании, лучше уточнить, чем угадывать